

TASK9: #51624

1. create an IAM user from the Root aws account.
2. set the policy or permission to the user, so that it can only access aws-Ec2 service.[Restricted privileges]
3. login as the IAM user and verify the working of the policies attached with the user.

NOTE: try the foolowing

1. IAM user with AWS management console access
2. IAM user with Programmatic access.

1. Create the IAM User (as Root)

- Log in to the AWS Management Console as the root user.
- Navigate to IAM → Users → Add users (Create user).
- Enter a username, Fadhil
- Select AWS credential type(s):
 - AWS Management Console access — set a console password.
 - Programmatic access — generates Access Key ID and Secret Access Key for CLI/SDK use.

2. Attach a Restricted EC2-Only Policy

Instead of using the broad AWS managed policy AmazonEC2FullAccess, a custom least-privilege JSON policy was created and attached via IAM → Policies → Create policy → JSON. The final policy restricts the user to EC2 actions only, and only within the ap-south-1 (Mumbai) region:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowEC2OnlyInMumbai",
      "Effect": "Allow",
      "Action": "ec2:*",
      "Resource": "*",
      "Condition": {
        "StringEquals": { "aws:RequestedRegion": "ap-south-1" }
      }
    },
    {
      "Sid": "DenyEC2OutsideMumbai",
      "Effect": "Deny",
      "Action": "ec2:*",
      "Resource": "*",
      "Condition": {
        "StringNotEquals": { "aws:RequestedRegion": "ap-south-1" }
      }
    },
    {
      "Sid": "DenyAllOtherServices",
      "Effect": "Deny",
      "NotAction": [
        "ec2:*",

```

```

        "iam:ChangePassword",
        "iam:GetAccountPasswordPolicy",
        "iam:GetUser"
    ],
    "Resource": "*"
}
]
}

```

Explanation of statements:

- AllowEC2OnlyInMumbai — allows all EC2 actions, but only when the request is made in ap-south-1.
- DenyEC2OutsideMumbai — explicit deny for EC2 actions in any other region (defense in depth; explicit Deny always overrides an Allow in IAM).
- DenyAllOtherServices — blocks every non-EC2 service (S3, Lambda, IAM, etc.), while allowing the user to manage their own console password (iam:ChangePassword, iam:GetAccountPasswordPolicy, iam:GetUser).

3. Verify Console Access

After the IAM user was created with console access, logging in required a forced password reset (AWS default behaviour when "User must create a new password at next sign-in" is checked). Initially this failed with "You may not be authorized to perform this action, or the new password does not comply with the account password policy," because the restrictive EC2-only policy blocked the iam:ChangePassword action. Once the password-related actions were explicitly excluded from the Deny statement (and the IAMUserChangePassword managed policy attached), the password reset succeeded.

Note: AWS IAM cannot restrict which region a console session opens in — the sign-in process is global and the region selector is always visible. The restriction instead blocks any EC2 action attempted outside ap-south-1, which achieves the same practical outcome. A direct bookmark to the Mumbai EC2 console can be shared for convenience:

```
https://ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1
```

4. Set Up and Verify Programmatic Access

Steps performed to generate and test Access Keys:

- Logged in as root/admin → IAM → Users → [user] → Security credentials tab.
- Under Access keys, selected Create access key, use case: Command Line Interface (CLI).
- Copied and securely stored the Access Key ID and Secret Access Key (shown only once).
- Installed AWS CLI v2 on Windows (via the official MSI installer, avoiding PowerShell execution-policy issues with the .ps1 installer).
- Configured the CLI with the generated credentials:

```

aws configure
AWS Access Key ID [None]: <Access Key ID>
AWS Secret Access Key [None]: <Secret Access Key>
Default region name [None]: ap-south-1
Default output format [None]: json

```

5. CLI Verification Results

Test	Command	Expected Result
EC2 in Mumbai	aws ec2 describe-instances --region ap-south-1	Success
EC2 outside Mumbai	aws ec2 describe-instances --region us-east-1	UnauthorizedOperation

Other service (any region)	aws s3 ls	AccessDenied
Launch instance in Mumbai	aws ec2 run-instances --region ap-south-1 ...	Success (valid AMI)
Launch instance elsewhere	aws ec2 run-instances --region us-east-1 ...	UnauthorizedOperation

OUTPUT:

```
PS C:\Users\U> aws --version
aws-cli/2.36.46 Python/3.14.6 Windows/11 script-exe/AMD64
PS C:\Users\U> aws configure

Tip: You can deliver temporary credentials to the AWS CLI using your AWS Console session by running the command 'aws log
in'.

AWS Access Key ID [None]: AKIASSMSW4VCZVI5GW7X
AWS Secret Access Key [None]: TiLM0lQjYcGJJyVTUzbhmpen8/A3Fil2g02REF1N
Default region name [None]: ap-south-1
Default output format [None]: json
PS C:\Users\U> aws ec2 describe-instances --region ap-south-1
{
  "Reservations": [
    {
      "ReservationId": "r-05ca3488c2871088b",
      "OwnerId": "176938214725",
      "Groups": [],
      "Instances": [
        {
          "Architecture": "x86_64",
          "BlockDeviceMappings": [
            {
              "DeviceName": "/dev/sda1",
              "Ebs": {
                "AttachTime": "2026-09-16T09:14:13+00:00",
                "DeleteOnTermination": true,
                "Status": "attached",
```

```
PS C:\Users\U> aws ec2 describe-instances --region us-east-1

aws: [ERROR]: An error occurred (UnauthorizedOperation) when calling the DescribeInstances operation: You are not author
ized to perform this operation. User: arn:aws:iam::176938214725:user/fadhil is not authorized to perform: ec2:DescribeIn
stances with an explicit deny in an identity-based policy: arn:aws:iam::176938214725:policy/IAMuser
PS C:\Users\U> |
```